

Audience: Internal / client showcase

Scope: Security-deposit lifecycle, fee collection, auto-refund automation, and Resend inbound forwarding

1. Deposit Strategy by Plan

Every transaction that includes a security deposit follows one of two strategies, assigned automatically by

Plan	Strategy

Pro	CHARGE_REFUND	14 days	Collected at charge, returned on refund
------------	---------------	---------	---

Enterprise	CHARGE_REFUND	30 days	Collected at charge, returned on refund
------------	---------------	---------	---

AUTHORIZATION_HOLD — *Stripe places a temporary hold on the customer's card. Money is never moved, and Stripe captures it.*

CHARGE_REFUND — *Stripe immediately charges the customer's card. An automatic refund is issued as soon as the vendor decides to keep the deposit.*

2. Fee Structure

Total cost to vendor on a captured deposit

Platform margin 0.5% (collected by Contrazy)

Total fee on capture = 2% + €0.25

Example – €1,000 deposit captured in full:

Stripe fee	£15.25
Platform fee	£5.00
Vendor receives	£979.75

3. Strategy A — AUTHORIZATION_HOLD (Starter)

How it works

The customer's card is pre-authorized. Stripe never moves money until the vendor triggers a capture. No platform fee is charged unless the deposit is captured.

```

sequenceDiagram
    participant Customer
    participant Stripe
    participant Database
    participant Vendor
    participant Platform

    Customer->>Stripe: 1 Card authorised (capture_method: manual)
    Stripe-->>Customer: 2 Hold placed on card
    Stripe->>Database: 3 status = AUTHORIZED
    Database->>Vendor: 4 Deposit hold active (dashboard)
    Note over Stripe: Hold window: 7 days
    alt [Vendor captures (keeps deposit)]
        Vendor->>Stripe: 5 paymentIntent.capture(application_fee_amount = 0.5%)
        Stripe->>Platform: 0.5% routed to platform
    end
  
```

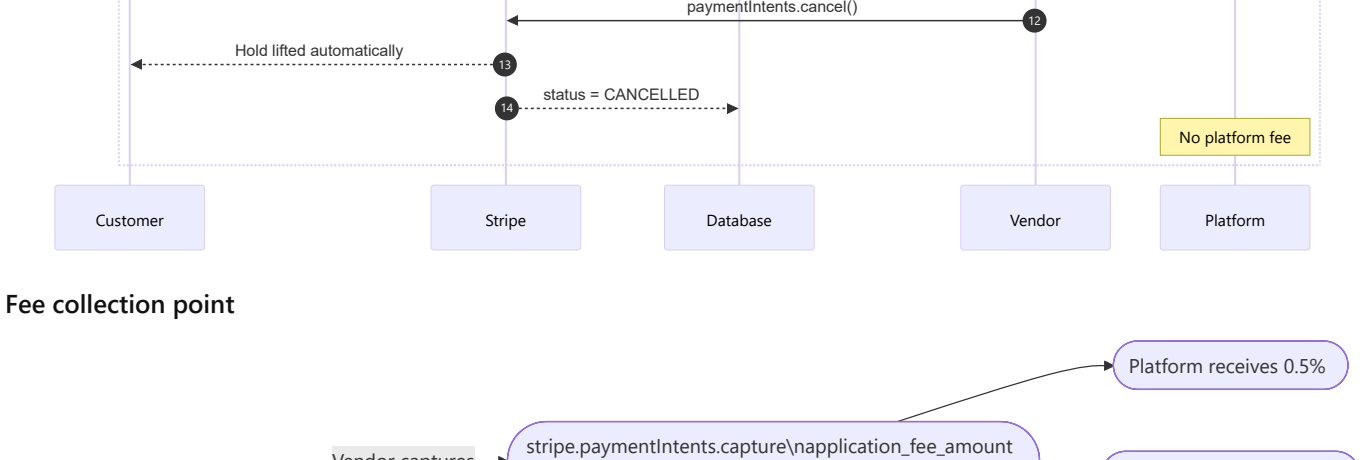
The diagram illustrates the flow of an authorization hold. It starts with a Customer authorizing a card (1), which Stripe places a hold on (2) and records as AUTHORIZED (3). The Database then shows the deposit hold as active (4). A 7-day hold window is indicated. If the Vendor captures the deposit (5), the payment intent is captured, and a 0.5% fee is routed to the Platform.

status = CAPTURED
[Vendor releases (return)]

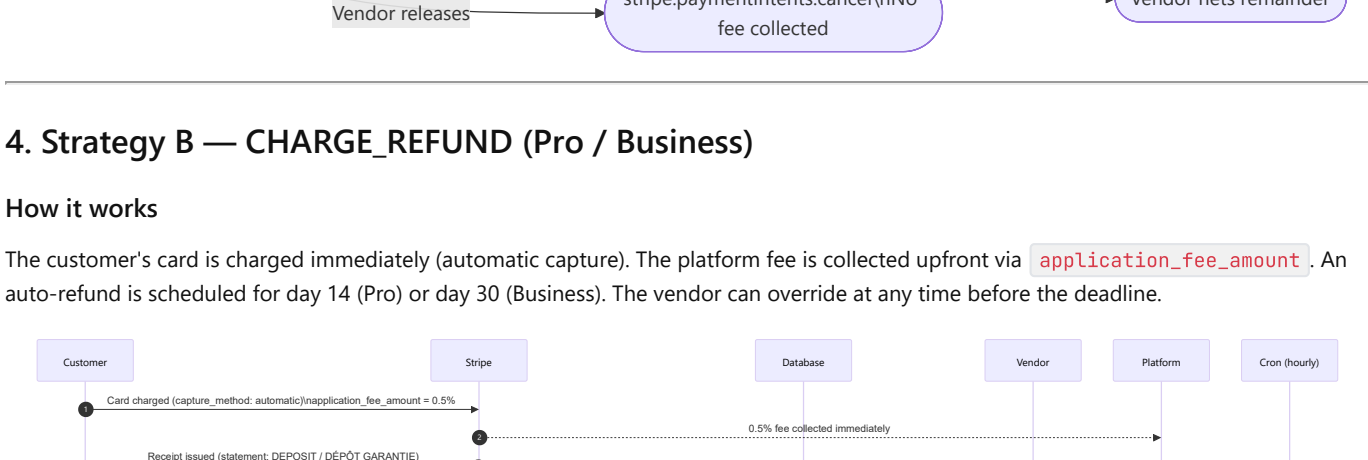
11

No platform fee

[Transaction cancelled]



Card authorised\nMoney moved

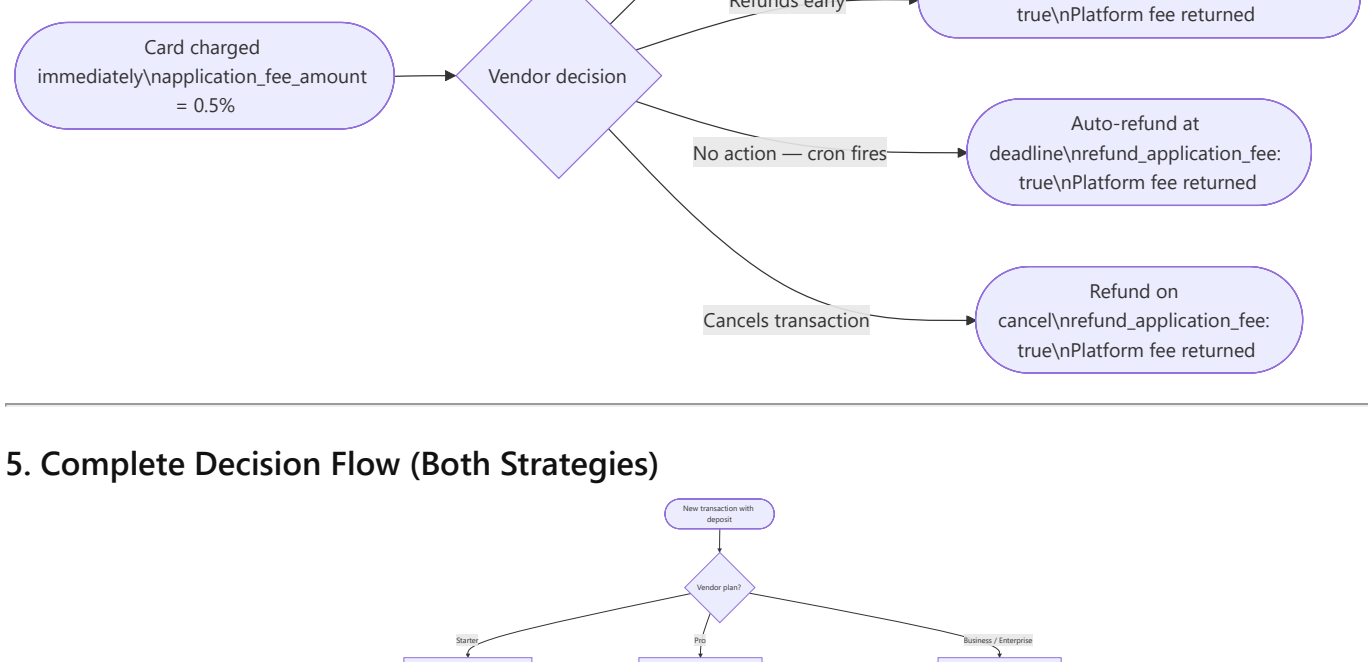
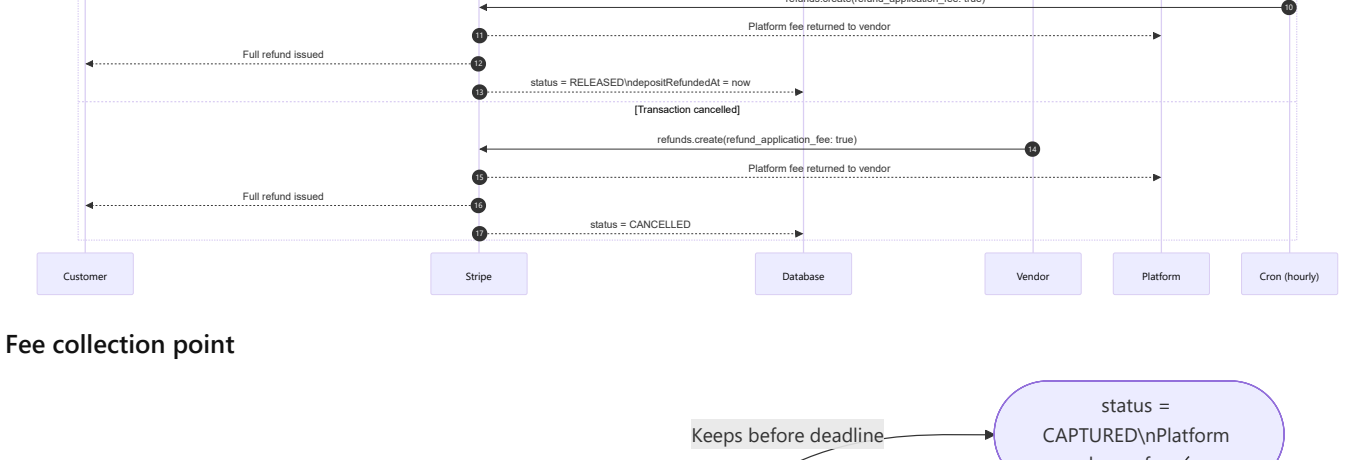


all

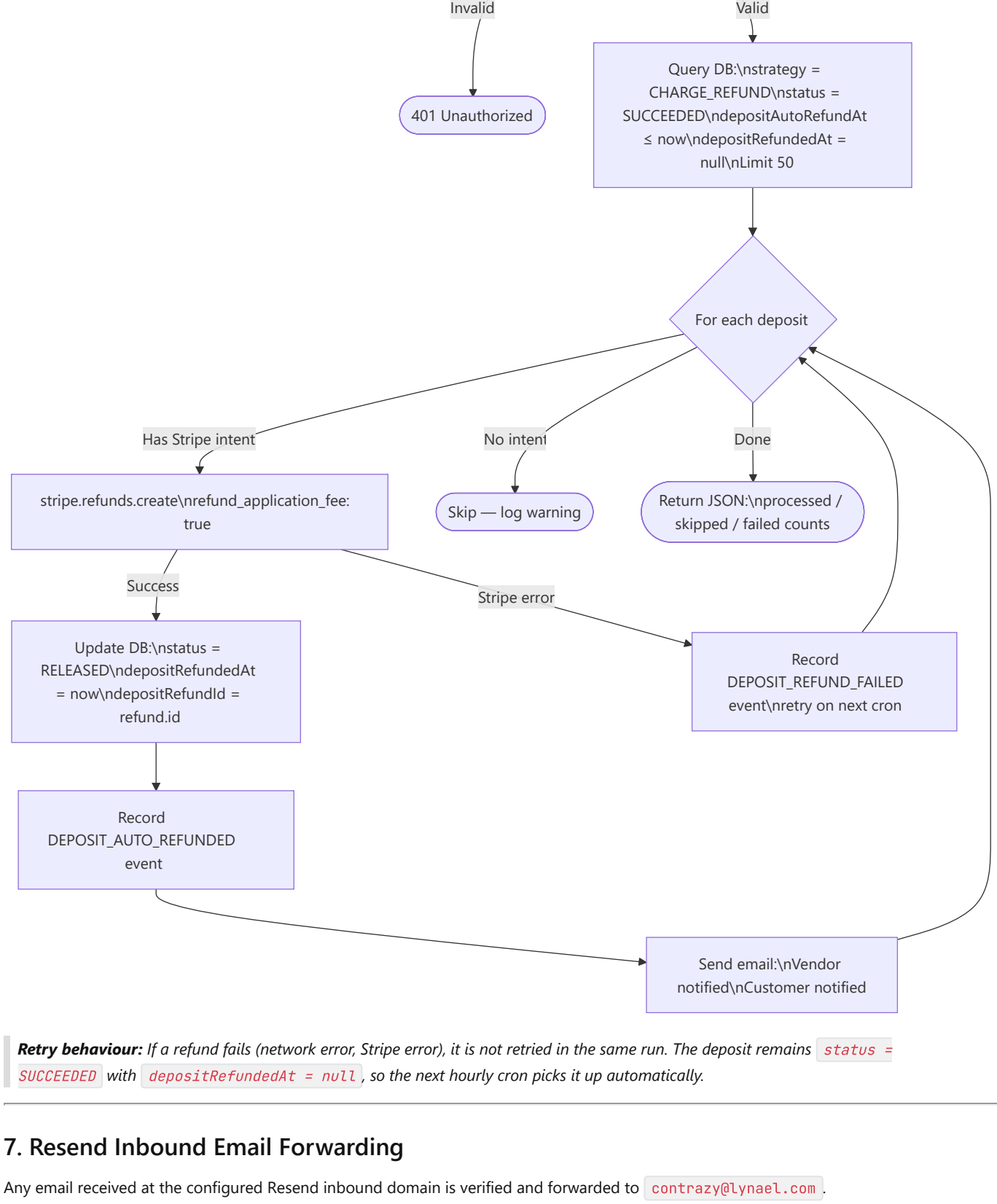
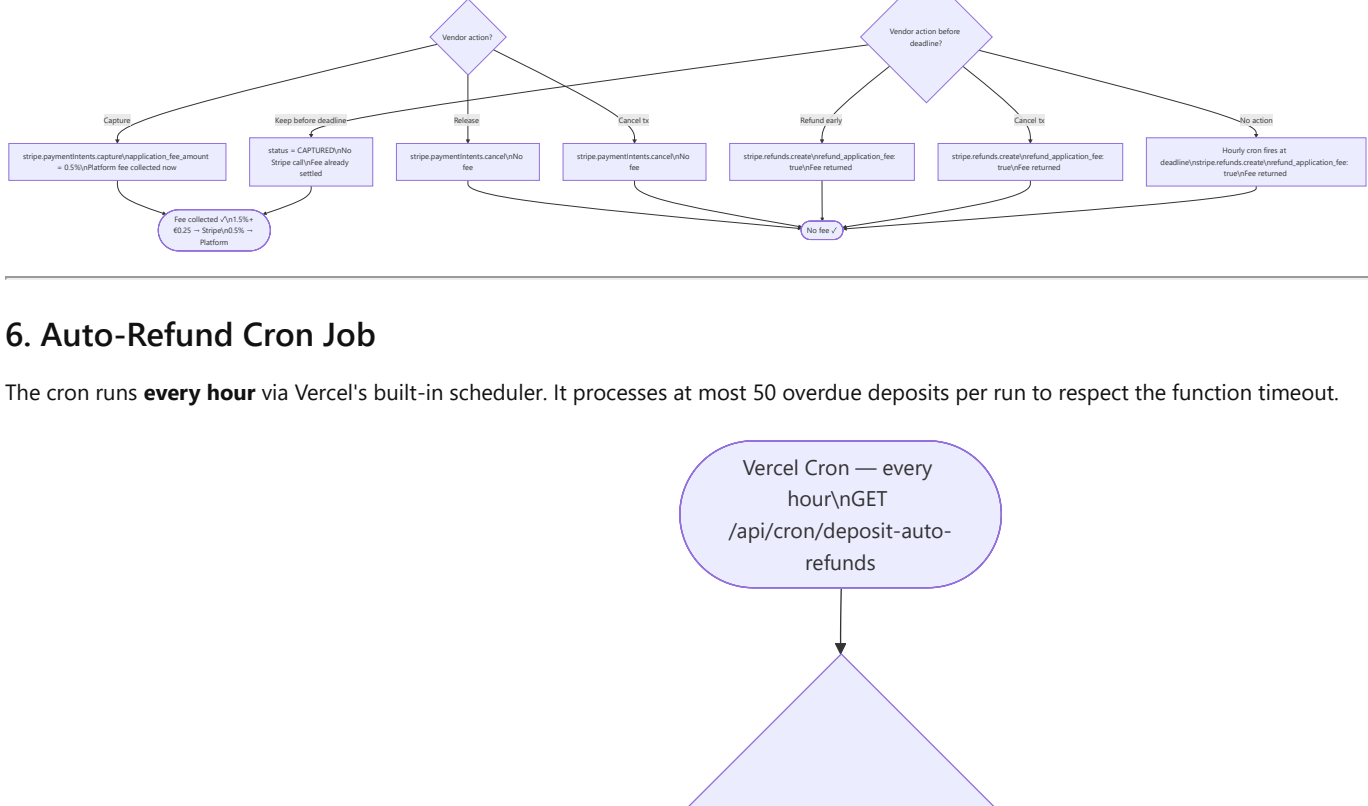
Vendor keeps deposit (before r

```

sequenceDiagram
    participant Vendor
    participant Platform
    participant Refund
    Vendor->>Platform: Full refund issued
    Platform->>Refund: Platform fee returned to vendor
    Refund->>Platform: status = RELEASED
    Platform->>Refund: [Auto-refund (deadline passed, error)]
  
```



Customer authorities
card/pic money moved yet

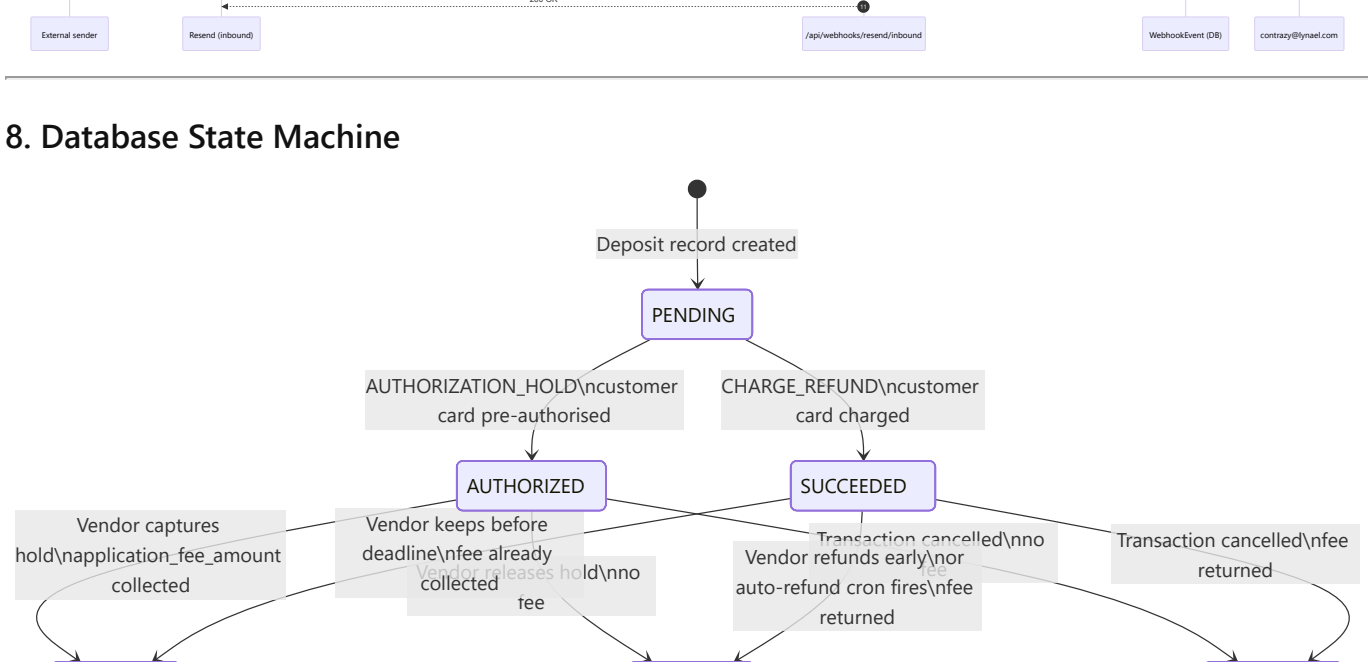
[illegible]

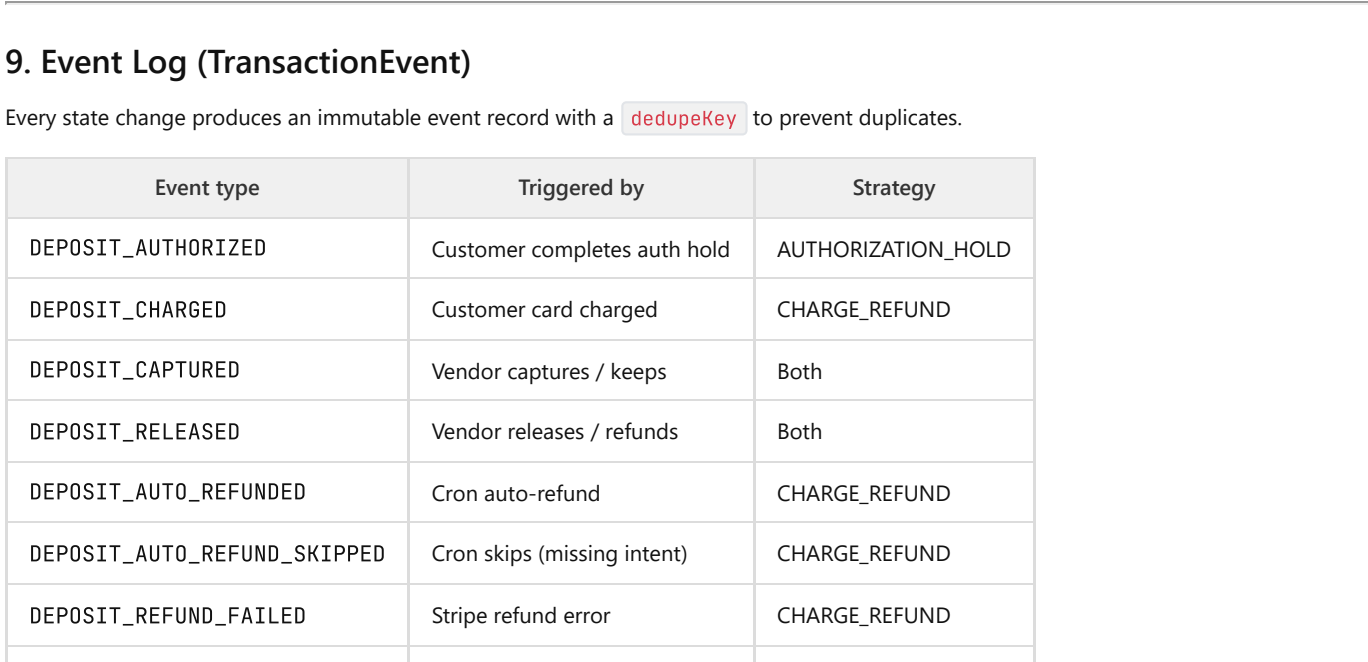
			(Ready process)
--	--	--	-----------------

```

1  thread_email_send(yield + REG32ND_INCLND_F0950AFD_T0V0x + REG32ND_AU2DT_BIC7_BMAA_optional[3rdArg] + PRIO_optional[3rdArg] + from | sender[]
2  Forwarded email delivered

```





10. Email Notifications

Trigger	Recipients	Term
---------	------------	------

Deposit authorized (hold)	Vendor	sendVendorDepositAlert
Deposit charged	Customer	sendDepositChargedEmail
Vendor captures	Vendor + Customer	sendVendorDepositStatusEmail + sendCustomerDepositStatusEmail
Vendor releases	Vendor + Customer	sendVendorDepositStatusEmail + sendCustomerDepositStatusEmail
Auto-refund	Customer	sendDepositAutoRefundedEmail
Inbound email received	contrazy@lynael.com	Forwarded via Resend

II. Environment Variables

```
# Deposit cron authentication
CRON_SECRET=$(cat /dev/urandom | tr -dc 'a-z0-9' | fold -w 32 | xargs echo | sha256sum | cut -d ' ' -f 1)

# Resend inbound email forwarding
RESEND_WEBHOOK_SECRET=whsec... # From Resend dashboard - Webhooks
RESEND_INBOUND_FORWARD_TO=contrazy@lynael.com
RESEND_AUDIT_BCC_EMAIL= # Optional - BCC on every forwarded email
```

Cron endpoint called by anyone	AUTH01-12a0
Second watch only endpoint	63c4UMAG-6

Duplicate webhook delivery	WebhookEvent unique constraint on (provider, providerEventId)
Double refund	Cron checks depositRefundedAt IS NULL before acting
Platform fee not collected on keep	application_fee_amount set at PaymentIntent creation for CHARGE_REFUND; Stripe guarantees it

Platform fee charged on release	refund_application_fee: true on every CHARGE_REFUND refund
Secrets in logs	No payment secrets, keys, or webhook bodies logged in production paths